

EasyDocs SaaS — Technical Blueprint

System Architecture, Project Context, and Phase 1 Development Roadmap

PROJECT NAME

EasyDocs Portal

ARCHITECTURE

Multi-Tenant SaaS Platform

CURRENT PHASE

Phase 1 (Foundation & Auth)

1. Project Aim & Vision

Primary Objective: Build a high-performance, secure, multi-tenant corporate document management platform that guarantees strict tenant data isolation, streamlined user onboarding via cryptographically secured invitations, and role-based access control across companies.

EasyDocs enables corporate organizations ("Companies") to securely host, categorize, and manage internal documentation. By employing a multi-tenant database structure anchored at the company level, EasyDocs guarantees that data is completely isolated, preventing cross-tenant leaks while allowing scalable employee account creation.

2. Technology Stack & Infrastructure

Core Technologies Engine:

Django Web Framework

Django REST Framework

SimpleJWT (JWT Auth)

React 18

React Router DOM v6

Tailwind CSS

PostgreSQL

3. System Architecture & Multi-Tenancy

EasyDocs enforces **logical data isolation** at the database and API query layer. Every primary data entity maintains an explicit relational link to a top-level Company record.

Core Architectural Rules:

- **Tenant Anchor:** The Company model acts as the primary tenant boundary.
- **UserProfile Association:** Django's default User model is extended via a UserProfile (One-to-One), linking users directly to a specific Company (Foreign Key).
- **Automated Query Filtering:** All backend document retrieval endpoints implicitly filter database queries using the active user's company_id obtained from the validated JWT claims.

4. Phase 1 Development Details

Phase 1 focuses on core infrastructure setup, database modeling, secure authentication, client-side routing, and tenant invitation mechanics.

Module	Implementation Scope	Status
Backend Core	Django REST Framework initialization, CORS configurations, SimpleJWT integration for refresh/access tokens.	COMPLETED
Data Models	Designing Company, UserProfile, Invitation, and preliminary Document database schemas.	COMPLETED
Frontend Shell	Vite React framework setup with Tailwind CSS integration, clean resolution of Tailwind linter classes, and layout scaffolding.	COMPLETED
Routing Architecture	Configuring createBrowserRouter with AuthProvider context wrapping and ProtectedRoutes layout guards.	COMPLETED
Invitation Engine	Secure UUID token generation for employee email invites, registration validation API, and account activation workflow.	IN PROGRESS
Document Vault	Role-based document upload, storage handler, and multi-tenant isolated list APIs.	PHASE 1 NEXT

5. Phase 1 Database Schema Blueprint

```
[Company]
├── id (UUID/PK)
├── name (CharField)
├── registration_date (DateTimeField)
└── is_active (BooleanField)

[UserProfile]
├── id (PK)
├── user (OneToOne -> Django User)
├── company (ForeignKey -> Company)
└── role (Choices: 'MANAGEMENT', 'EMPLOYEE')

[Invitation]
├── id (UUID/PK)
├── email (EmailField)
├── company (ForeignKey -> Company)
├── token (UUIDField, Unique)
├── is_used (BooleanField)
└── expires_at (DateTimeField)
```

6. Frontend Routing & Auth State Architecture

The React application utilizes React Router v6 object-based router configuration (createBrowserRouter). Standard layout encapsulation ensures global auth context is accessible everywhere:

```
App Tree Structure:
```

```
└─ Public: /login, /registration
└─ Protected (ProtectedRoutes):
    └─ Layout ():
        └─ / (Dashboard Index)
        └─ /dashboard
```